WebXR domain relocation and core benchmark reproduction

```
\documentclass[11pt]{article}
\usepackage[margin=25mm]{geometry}
\usepackage[T1]{fontenc}
\usepackage{booktabs}
\usepackage{hyperref}
\title{WebXR domain relocation and core benchmark reproduction}
\author{Codex}
\date{Created-at: 2026-09-04 18:03 CEST\\ETA: 4--6 hours}
\begin{document}
\maketitle

\section{Authority and objective}
\texttt{APPROVED-CORE-EDIT: relocate-webxr-to-transformation}\\
\texttt{APPROVED-CORE-EDIT: repair-core-yaml-syntax}

Move the complete, latest WebXR semantic implementation from
\texttt{hymeko/core} to the downstream \texttt{hymeko/transformation}
workspace.  The core retains only generic IR, identifiers, resolution,
canonical hashing, wire codecs and numerical primitives.  The migration must
not create a second IR or change WebXR behaviour.  After the move, reproduce
the selected published core benchmark cells from clean, pinned commits.

\section{Measured starting state}
The committed core contains \texttt{hymeko\_core::webxr}, its integration
tests, three \texttt{data/webxr} fixtures and a public module export.  The user
checkout additionally contains the uncommitted query, camera/fiducial
vocabulary extension and Maxwell demo.  Those files are the migration source;
the contract branch itself remains untouched.

The transformation workspace already owns IR-to-domain projections.  Its
\texttt{hymeko\_geometry} crate is the generic geometric reading, while
\texttt{hymeko\_query} is explicitly domain-free.  WebXR therefore becomes a
separate \texttt{hymeko\_webxr} consumer crate, depending on the canonical
\texttt{hymeko\_core} IR and not vice versa.

\section{Phases}
\begin{enumerate}
  \item Repair only the YAML syntax of \texttt{CORE.YAML}; preserve meaning.
  \item Add \texttt{hymeko\_webxr} to the transformation workspace.  Port the
        latest validator, borrowed query, fixtures and tests by changing only
        crate paths and fixture roots.
  \item Prove source compilation, exact validation errors, query results,
        metamorphic invariance and the Maxwell/fiducial fixture in transformation.
  \item Remove the WebXR export, implementation, tests and fixtures from core;
        regenerate the content-addressed corpus golden from the reduced corpus.
  \item Correct the platform ownership record.  Historical reports remain
        evidence and are marked superseded rather than rewritten as history.
  \item Run focused tests, full workspace tests, formatting and clippy in both
        repositories.  Verify one-way dependency: transformation $\rightarrow$
        core, never core $\rightarrow$ transformation.
  \item On a clean core worktree, reproduce the cached benchmark cells with
        pinned compiler, commit, command, workload digest, warm/cold policy,
        median, IQR and worst value.  Any difference above 10\% is investigated
        before a claim is accepted.
\end{enumerate}

\section{File ownership}
\begin{tabular}{@{}lll@{}}
\toprule
Concept & Destination & Core remainder \\
\midrule
Canonical IR and accessors & \texttt{hymeko\_core} & authoritative \\
Spatial/WebXR vocabulary & \texttt{hymeko\_webxr/data} & none \\
WebXR validation/query & \texttt{hymeko\_webxr} & none \\
Browser runtime handles & downstream endpoint & none \\
Historical measurements & originating reports & marked superseded \\
\bottomrule
\end{tabular}

\section{Gates and rollback}
No runtime WebXR handle enters the IR.  No core API is added to support the
move unless a missing generic accessor is proven.  The transformation branch
must pass before core deletion is committed.  Rollback is dropping the two
feature branches; the original dirty files remain preserved in the user
checkout until both branches are verified and pushed.

Peak RSS is capped at 16 GiB.  Long benchmark commands emit progress.  No
benchmark is run concurrently with another compiler or benchmark process.
\end{document}
```